Contrato de API · arce.dev

Oct 26, 2026 · @Ezequiel

Este documento fija cómo hablan el cliente (B y D) y la API (C): URL, formato de respuestas, permisos y los cambios respecto del plan del 13/10. Cuando algo difiera del plan, manda este documento.

Estado al 3/11: la API de `server/` está mergeada en `main`, publicada en Render y conectada al frontend de Vercel. 271 tests pasan y el smoke contra lo publicado también. Están probados contra MongoDB Atlas (seed cargado), Cloudinary y el login con contraseña. Falta probar con credenciales reales el envío de emails (Brevo) y el login con Google. **La sección 0 manda sobre el resto del documento**: las secciones 1 a 10 son la versión original y algunos ejemplos (formato de errores, listas, nombres de campos) cambiaron al adaptar la API al frontend.

Revisión de seguridad (27/10): se probaron ataques contra todos los endpoints y se corrigieron 16 problemas, dos de ellos altos: las sesiones no se cerraban al cambiar la contraseña y se podía secuestrar una cuenta registrando su email antes de que su dueño entrara con Google. Los cambios que tocan al cliente están en la sección 10.

0. Actualización del 3/11: la API quedó adaptada al frontend

Esta sección manda sobre el resto cuando haya diferencias. API publicada: `https://portfolio-fullstack-cidz.onrender.com` (Render gratis: si estaba dormida, la primera petición tarda cerca de un minuto). Sitio: `https://portfolio-fullstack-kappa-wine.vercel.app`. El frontend usa `client/src/services/api.js`; la API se adaptó a lo que él ya manda y espera.

Formato de respuestas

- Éxito: `{ ok: true, data }`.
- Error: `{ ok: false, error: "mensaje", code: "validacion", details: [{ field, message }] }`. `error` es un **texto** (no un objeto), `details` aparece solo en errores por campo y `field` es el nombre del campo del formulario. `code` es un identificador estable; algunos errores suman `attemptsLeft` o `retryAfterSeconds` al lado. Esto reemplaza el formato `{ error: { code, message, fields } }` de las secciones 1 y 2.
- Todo objeto trae `_id` (lo que lee el frontend) e `id` (mismo valor).
- Listas de usuarios y consultas: `data: { items, page, limit, total, pages }`. La de proyectos es un array con `meta: { page, limit, total, totalPages }`.

Qué manda y espera cada pantalla

Pantalla	Pedido	Notas
Contacto	`POST /inquiries` con `nombre, email, telefono, tipo, presupuesto, asunto, mensaje` y el archivo en `adjunto` (multipart)	`presupuesto` (chico, medio o grande) es obligatorio en cotización. Adjunto PDF, JPG o PNG de hasta 2 MB. Los errores vuelven con esos mismos nombres.
Registro	`POST /auth/register` con `nombre, email, password`	No pide repetir la contraseña ni aceptar términos (si vienen, se validan).
Mi cuenta	`PATCH /auth/me` con `{ name }` o `{ password, currentPassword }`; `DELETE /auth/me` sin cuerpo; `GET /inquiries/mine`	Cambiar la contraseña **exige la actual** si la cuenta ya tiene una; quien entró solo con Google crea la primera sin ella. La respuesta trae un `token` nuevo que el cliente tiene que guardar (las demás sesiones se cierran).
Recuperar	`POST /auth/forgot { email }` y `POST /auth/reset { token, password }`	El enlace del email es `/recuperar?token=…`.
Proyectos (admin)	`POST` y `PUT /projects` en multipart: `title, summary, description, demoUrl, repoUrl, featured, visible` (como texto), `stack` repetido, `keepImages` (URLs a conservar) e `images` (archivos)	Hasta 5 imágenes JPG, PNG o WebP de 2 MB; se suben a Cloudinary. También se acepta JSON. `GET /projects?all=true` le da al admin todos, con borradores y detalle. `images` vuelve como lista de URLs.
Servicios (admin)	`name, description, icon, color, priceFrom, order, visible`	`icon`: code, server, database o chat. `color`: carmin, negro, vicuna, exito u oro (no hex). `order` 0 a 999. `GET /services` le muestra al admin también los ocultos. Reordenar = dos `PUT` que intercambian `order`.

Pantalla	Pedido	Notas
Perfil (admin)	`GET` y `PUT /profile`: `name, title, tagline, bio, email, location, github, linkedin, stack, experience` y la foto en `photo` (multipart)	`tagline` reemplaza a `phrase`. `experience` va como JSON. Sin foto nueva se conserva la anterior.
Usuarios (admin)	`GET /users?role&q&page`, `PATCH /users/:id { role, active }`	Un admin no puede cambiarse el rol ni desactivarse. `DELETE` sigue existiendo.
Bandeja del operador	`GET /inquiries?tipo&estado&desde&hasta&q&page`	Cada fila trae `message`, `phone`, `budget`, `attachment` y `replies` (con `operator.name`), porque el panel despliega la fila sin pedir el detalle.
Estadísticas (admin)	`desde, hasta, agrupar, tipo`	`summary`: `{ visits, users, newUsersThisMonth, logins, inquiries }` (números). `visits`: `[{ fecha, total }]`. `inquiries-by-type`: `[{ tipo, total }]`. `registrations`: `[{ semana, fecha, total }]`.

Decisiones nuevas

- **Vincular Google no borra la contraseña:** las dos formas de entrar conviven. Al vincular se cierran las sesiones abiertas.
- **Eliminar una cuenta anonimiza sus consultas** en vez de borrarlas: nombre "Cuenta eliminada", email `eliminada@anonimo.invalid`, sin teléfono, adjunto ni vínculo con la cuenta. Siguen en la bandeja y en las estadísticas, y no se pueden responder (409 `consulta_anonima`). Los accesos y las visitas quedan sin dueño.
- **Cambiar la contraseña** exige la actual (salvo cuentas solo de Google) y cierra las otras sesiones.
- **Emails** por SMTP con Brevo, puerto 2525 (Render gratis bloquea 25, 465 y 587). En producción, sin SMTP configurado el envío falla a propósito.
- **Base de datos:** 9 colecciones (`users`, `logins`, `loginattempts`, `projects`, `services`, `profile`, `inquiries`, `visits`, `counters`). Las respuestas del operador van anidadas dentro de cada consulta.

Qué falta

- Probar con credenciales reales los emails (Brevo): confirmación de la consulta, respuesta del operador y recuperar contraseña.
- Login con Google: el servidor está listo; el botón del sitio dice "Disponible pronto".

1. Convenciones y decisiones

- **Base URL:** `VITE_API_URL` + `/api` (por ejemplo `http://localhost:4000/api`). Todo viaja como JSON, salvo las subidas de archivos (`multipart/form-data`).
- **Autenticación:** `Authorization: Bearer <JWT>`. El token dura 7 días y el cliente lo guarda en `localStorage`. Si vence o es inválido, la API responde 401 y el cliente manda al login.
- **Fechas:** ISO 8601 en UTC (`2026-10-15T23:12:00.000Z`). El formato `15/10/2026 · 23:12` lo arma el cliente.
- **Ids:** string de 24 caracteres (`_id` de Mongo, llega como `id`). Las consultas además tienen `number`, el «N.º 0031» de los emails.
- **Campos en inglés, textos en español:** los nombres de campo y los valores de enums siguen el plan (`type: "cotizacion"`, `status: "en_proceso"`). Las etiquetas visibles («Cotización», «En proceso») las pone el cliente.

Respuestas correctas

```
{ "ok": true, "data": { "id": "6710f0c2a1b2c3d4e5f60718", "title": "Gestor de turnos" } }
```

Los listados agregan `meta` y siempre se pueden paginar con `?page=1&limit=20` (límite máximo 100):

```
{ "ok": true, "data": [ ], "meta": { "page": 1, "limit": 20, "total": 30, "totalPages": 2 } }
```

Quién puede qué

Nivel	Rol necesario	Notas
Público	ninguno	Visitante sin sesión.
Registrado	`user`, `operator` o `admin`	Cualquier cuenta activa.
Operador	`operator` o `admin`	El admin también entra a `/operador`.

Nivel	Rol necesario	Notas
Admin	`admin`	Solo administradores.

Decisiones tomadas (cambian el plan)

#	Decisión	Afecta a
1	Las reglas de validación son las del prototipo, no las del plan (sección 2).	B, D
2	Una cuenta desactivada no puede iniciar sesión: la API responde 403 `cuenta_desactivada`.	B, D
3	El admin accede a `/operador`; el texto «cada rol ve solo su panel» del manual hay que corregirlo.	A
4	Las imágenes y adjuntos se suben con `POST /api/uploads` (Cloudinary) y los formularios envían solo la URL.	B, D
5	La recuperación de contraseña usa un enlace por email que vence en 1 hora.	B
6	Tras 5 logins fallidos en 15 minutos para un mismo email desde la misma IP, la API responde 429 y bloquea esa combinación 15 minutos. 50 fallos contra una cuenta, desde cualquier IP, la bloquean para todos.	B
7	El proyecto reemplaza `description` por `problem` y `solution`; el servicio reemplaza `icon` por `color`.	D
8	La API guarda los datos del perfil público (`profile`) y el cliente los lee; ya no van fijos en el código.	A, B, D
9	El admin no puede desactivarse, quitarse el rol ni borrarse a sí mismo, y nunca puede quedar el sistema sin un admin activo.	D
10	El listado público de proyectos no devuelve `demoUrl`, `repoUrl`, `problem`, `solution` ni `highlights`: eso solo lo ve un usuario registrado.	B

#	Decisión	Afecta a
11	GET y PATCH /api/auth/me devuelven el usuario directamente en data, sin envolverlo en { user }.	B
12	El listado público de proyectos viene con los destacados primero y después los más nuevos.	B
13	Eliminar una cuenta (la propia o desde Usuarios) borra también sus consultas y sus accesos. Las visitas se conservan, sin dueño.	B, D
14	Límites de pedidos, por IP salvo que se diga otra cosa: 300 por minuto en toda la API; 5 consultas por hora y 3 por hora por cada email destinatario; 3 subidas de adjuntos por hora (todos menos el admin); 60 visitas por minuto; 20 registros por hora; 20 logins fallidos y 30 intentos de Google cada 15 minutos; recuperación de contraseña 10 por hora y 3 por hora por email; 20 intentos de reset por hora. Pasados, la API responde 429.	B

2. Errores y validación

Toda respuesta con error trae un código estable para el programa y un mensaje en voseo listo para mostrar. Los errores de validación traen además un mensaje por campo, para pintarlo junto al input sin borrar lo que el usuario escribió.

```
{
  "ok": false,
  "error": {
    "code": "validacion",
    "message": "Revisá los campos marcados.",
    "fields": {
      "email": "Falta el dominio, por ejemplo martin@correo.com",
      "message": "Mínimo 20 caracteres (llevás 8)."
    }
  }
}
```

HTTP	`error.code`	Cuándo
400	`validacion`	Algún campo no cumple las reglas; trae `fields`.

HTTP	`error.code`	Cuándo
401	`no_autenticado`	Falta el token o venció. El cliente manda al login y vuelve a la página pedida.
401	`credenciales_invalidas`	Email o contraseña incorrectos. Trae `attemptsLeft` (de 5).
403	`sin_permiso`	El rol no alcanza para este endpoint.
403	`cuenta_desactivada`	La cuenta existe pero un admin la desactivó.
404	`no_encontrado`	El recurso no existe, o no es visible para este usuario.
409	`email_en_uso`	Registro con un email que ya tiene cuenta: «Ese email ya tiene una cuenta. ¿Querés ingresar?»
409	`nombre_en_uso`	Servicio con un nombre repetido.
413	`archivo_pesado`	El archivo supera 2 MB.
415	`formato_no_permitido`	Tipo de archivo fuera de la lista.
429	`demasiados_intentos`	Límite de intentos; trae `retryAfterSeconds`.
500	`error_interno`	Falla inesperada. El mensaje nunca incluye detalles técnicos.
400	token_invalido	El enlace de recuperación venció o ya se usó.
413	pedido_muy_grande	Un pedido JSON supera los 100 KB.
502	email_no_enviado	La respuesta del operador no pudo enviarse por email. No se guardó nada y se puede reintentar.
502	almacenamiento_error	Cloudinary falló al guardar un archivo.
503	almacenamiento_no_configurado	Al servidor le faltan las credenciales de Cloudinary: no se puede subir nada.
503	google_no_configurado	Al servidor le falta GOOGLE_CLIENT_ID: el login con Google no está disponible.
400	pedido_invalido	El pedido no se pudo leer, por ejemplo una URL mal codificada como /api/projects/%. Lo devuelve cualquier otro 4xx del propio Express.

Los errores de archivo también traen `fields` (por ejemplo `file` o `attachment`) para que el cliente los muestre junto al selector.

Un JSON mal formado responde 400 `validacion` con el mensaje «El pedido no tiene un JSON válido.» y `fields` vacío. Un id con formato inválido (por ejemplo `/api/projects/abc`) responde 404 `no_encontrado`, igual que uno que no existe. Los 409 traen `fields` con el mismo mensaje, para pintarlo junto al campo, y los 429 traen `retryAfterSeconds`.

Reglas de validación (valen en el front y en la API)

La API valida siempre, aunque el cliente ya lo haya hecho. Todos los textos se recortan (`trim`) antes de validar y se les quitan etiquetas HTML.

Campo	Regla	Mensaje
Nombre (registro)	3 a 60 caracteres, solo letras (con tildes y ñ), espacios y `'`; debe llevar nombre y apellido	«Ingresá tu nombre y apellido.» · «Mínimo 3 letras.» · «Usá solo letras y espacios.» · «Escribí nombre y apellido.»
Nombre (contacto, cuenta, usuario)	Igual, sin exigir apellido	«Ingresá tu nombre.» · «Mínimo 3 letras.» · «Usá solo letras y espacios.»
Email	Se pasa a minúsculas; formato `algo@dominio.tld` con tld de 2 letras o más	«Ingresá tu email.» · «Falta la @, por ejemplo martin@correo.com» · «Falta el dominio, por ejemplo martin@correo.com» · «El email no es válido. Ejemplo: nombre@correo.com»
Contraseña	8 a 72 caracteres, con mayúscula, minúscula y número	«La contraseña debe tener 8 caracteres o más, una mayúscula, una minúscula y un número.»
Repetir contraseña	Igual a la contraseña (la API lo recibe como `passwordConfirm`)	«Las contraseñas no coinciden.»
Teléfono	Opcional; exactamente 10 dígitos	«Solo números, sin espacios ni guiones.» · «Tienen que ser 10 dígitos (código de área + número).»
Tipo de consulta	`general`, `cotizacion` o `asesoria`	«Elegí el tipo de consulta.»

Campo	Regla	Mensaje
Presupuesto	Obligatorio solo en `cotizacion`: `chico`, `medio` o `grande`	«Elegí un rango de presupuesto.»
Asunto	5 a 120 caracteres	«Escribí un asunto.» · «El asunto es muy corto (mínimo 5 caracteres).»
Mensaje	20 a 1000 caracteres	«Contanos qué necesitás.» · «Mínimo 20 caracteres (llevás N).»
Adjunto de consulta	PDF, JPG o PNG; hasta 2 MB	«Formato no permitido: solo PDF, JPG o PNG.» · «El archivo pesa X MB. El máximo es 2 MB.»
Respuesta del operador	20 a 5000 caracteres	«Escribí la respuesta antes de enviar.» · «La respuesta es muy corta (mínimo 20 caracteres).»
Proyecto: título	3 a 80 caracteres	«Escribí un título de al menos 3 caracteres.»
Proyecto: resumen	Obligatorio, hasta 200	«El resumen es obligatorio.» · «Máximo 200 caracteres.»
Proyecto: stack	Al menos 1 tecnología, hasta 12, cada una de 1 a 24 caracteres	«Indicá al menos una tecnología.»
Proyecto: demo y repo	Opcionales; deben empezar con `https://`	«Tiene que empezar con https://»
Proyecto: imágenes	Hasta 5; JPG, PNG o WebP; hasta 2 MB cada una	«Formato no permitido: solo JPG, PNG o WebP.» · «La imagen pesa X MB. El máximo es 2 MB.»
Servicio: nombre	3 a 60 caracteres; único sin distinguir mayúsculas	«Escribí un nombre de al menos 3 caracteres.» · «Ya existe un servicio con ese nombre.»
Servicio: descripción	Obligatoria, hasta 300	«La descripción es obligatoria.» · «Máximo 300 caracteres.»

Campo	Regla	Mensaje
Servicio: precio	Opcional; entero ⩾ 0, sin decimales	«Solo números, sin puntos ni signo $.»
Perfil: nombre	3 a 60, solo letras y espacios	«El nombre es obligatorio.» · «Usá solo letras y espacios.»
Perfil: título	5 a 60 caracteres	«Mínimo 5 caracteres.»
Perfil: frase	1 a 60 caracteres	«La frase tiene que tener entre 1 y 60 caracteres.»
Perfil: presentación	40 a 400 caracteres	«Mínimo 40 caracteres.»
Perfil: ubicación	3 a 60 caracteres	«Mínimo 3 caracteres.»
Perfil: GitHub	`https://github.com/usuario`	«Tiene que ser https://github.com/tu-usuario»
Perfil: LinkedIn	`https://linkedin.com/in/usuario`	«Tiene que ser https://linkedin.com/in/tu-usuario»
Perfil: stack	Hasta 16 tecnologías de 1 a 24 caracteres, sin repetir	«Ya está en el stack.» · «Máximo 16 tecnologías.»

El asunto máximo de 120 caracteres es una regla nueva que el prototipo no tenía: hay que agregarla al formulario de contacto.

Detalles que se sumaron al implementar

- **Email:** los mensajes «Falta la @…» y «Falta el dominio…» salen solo en el formulario de contacto. Registro y login usan siempre «El email no es válido. Ejemplo: nombre@correo.com». El email no puede tener comas, punto y coma, comillas, paréntesis ni <>, y llega hasta 254 caracteres.
- **Nombre y asunto de la consulta:** los saltos de línea y los espacios repetidos se juntan en uno, para que nada llegue como encabezado de un email.
- **Contraseña:** además de las reglas, hasta 72 caracteres («La contraseña puede tener hasta 72 caracteres.»).
- **Adjunto de la consulta:** tiene que haberse subido con `POST /api/uploads`. Una URL de otro sitio, un peso mayor a 2 MB o un tipo fuera de PDF, JPG y PNG responden «El adjunto no es válido. Subilo de nuevo.» en `fields.attachment`.

- **Proyecto:** más de 6 puntos técnicos, «Máximo 6 puntos.»; más de 12 tecnologías, «Máximo 12 tecnologías.»; una imagen sin `url` https o sin `publicId`, «Cada imagen necesita url (https) y publicId.». Las tecnologías repetidas se descartan en silencio.
- **Servicio:** un color fuera de la paleta, «Elegí uno de los colores de la paleta.»; un precio mayor a 100.000.000, «El precio es demasiado alto.». El nombre es único sin distinguir mayúsculas ni tildes.
- **Perfil:** GitHub y LinkedIn son opcionales; vacío significa sin enlace. Cada puesto de experiencia necesita período (hasta 40 caracteres) y cargo (hasta 100); el detalle admite 200. Cada tecnología admite hasta 24 caracteres.
- **Filtros de los listados:** «Tipo no válido.», «Estado no válido.», «Rol no válido.», «Fecha no válida. Usá AAAA-MM-DD.», «La fecha final tiene que ser igual o posterior a la inicial.», «El rango máximo es de 366 días.» y, en estadísticas, «Agrupar tiene que ser dia o semana.».
- Los campos que no figuran en este contrato se ignoran.

3. Modelos de datos ajustados

Son nueve colecciones: las cinco del plan más `profile` y `logins`, y dos internas: counters (numera las consultas) y loginattempts (los intentos de login fallidos; Mongo los borra solos a los 15 minutos). `createdAt` y `updatedAt` los pone Mongoose en todas. Lo que cambia respecto del plan está en **negrita**.

users

Campo	Tipo	Reglas
`name`	String	Ver sección 2. Si viene de Google, se limpia en vez de rechazarse.
`email`	String	Único, minúsculas.
`passwordHash`	String o null	bcrypt. Nunca sale en una respuesta. Null si la cuenta solo usa Google.
`googleId`	String o null	Reemplaza a `provider`. Una cuenta puede tener contraseña, Google o las dos.
`role`	String	`user` (por defecto), `operator` o `admin`.
`active`	Boolean	`true` por defecto. Si es `false`, no puede iniciar sesión.

Campo	Tipo	Reglas
`loginCount`, `lastLogin`	Number, Date	Se actualizan en cada login correcto.
`newsletter`	Boolean	La casilla «quiero recibir un email cuando haya proyectos nuevos». Solo se guarda; no se envía nada.
`termsAcceptedAt`	Date	Se completa al registrarse.
`passwordReset`	`{ tokenHash, expiresAt }`	Para recuperar la contraseña. Se borra al usarse.

En las respuestas, el usuario se ve así. `providers` reemplaza al `provider` del plan y puede tener `"email"`, `"google"` o ambos:

```
{
  "id": "6710f0c2a1b2c3d4e5f60718",
  "name": "Valentina Ríos",
  "email": "valentina@correo.com",
  "role": "user",
  "active": true,
  "providers": ["google"],
  "newsletter": true,
  "loginCount": 6,
  "lastLogin": "2026-10-25T22:41:00.000Z",
  "createdAt": "2026-10-14T13:05:00.000Z"
}
```

logins (nueva)

Campo	Tipo	Reglas
`userId`	ObjectId	Quien inició sesión.
`method`	String	`password` o `google`.
`at`	Date	Con índice. Es lo que permite contar «accesos autenticados» en un período, algo que `loginCount` no puede.

projects

Campo	Tipo	Reglas
`title`	String	3 a 80.
`summary`	String	Hasta 200. Es lo único de texto que ve un visitante.
`problem`, **`solution`**	String	Reemplazan a `description`. Son los bloques «El problema» y «La solución» del detalle. Hasta 1000 cada uno.
`highlights`	[String]	Los puntos técnicos del detalle. Hasta 6, de 160 caracteres cada uno.
`role`, **`duration`**	String	Por ejemplo «Diseño, backend y frontend» y «8 semanas». Hasta 80 y 40 caracteres.
`stack`	[String]	Al menos 1, hasta 12. El cliente convierte el texto con comas en una lista.
`images`	[{ `url`, `publicId` }]	Hasta 5. La primera es la portada.
`demoUrl`, `repoUrl`	String o null	`https://`. Opcionales.
`visible`	Boolean	`false` por defecto: un proyecto nuevo queda como borrador.
`featured`	Boolean	Se mantiene del plan; ninguna pantalla lo usa todavía.

services

Campo	Tipo	Reglas
`name`	String	3 a 60, único sin distinguir mayúsculas ni tildes.
`description`	String	Hasta 300.
`color`	String	Reemplaza a `icon`. Uno de `#9B1B22`, `#160F10`, `#8A5A35`, `#4F6B3E`, `#7A5A12`.
`priceFrom`	Number o null	Entero ≥ 0. Null significa «a consultar».
`order`	Number	Posición de aparición, desde 1, sin repetir.
`visible`	Boolean	`false` por defecto.

profile (nueva, un solo documento)

Campo	Tipo	Reglas
`name`, `title`, `phrase`, `bio`	String	Ver sección 2.
`email`, `location`	String	Email válido; ubicación de 3 a 60.
`github`, `linkedin`	String	Opcionales: si faltan, quedan en null. URL con el formato de la sección 2.
`photo`	{ `url`, `publicId` } o null	JPG, PNG o WebP, hasta 2 MB.
`stack`	[String]	Hasta 16, de 1 a 24 caracteres, sin repetir.
`experience`	[{ `period`, `role`, `detail` }]	Hasta 10. Textos de hasta 40, 100 y 200 caracteres.

inquiries

Campo	Tipo	Reglas
`number`	Number	Único y correlativo (31, 32…). El cliente lo muestra con ceros: «N.º 0031».

Campo	Tipo	Reglas
`name`, `email`, `phone`	String	Ver sección 2. El teléfono es opcional.
`type`	String	`general`, `cotizacion` o `asesoria`.
`budget`	String o null	`chico` (hasta $500.000), `medio` ($500.000 a $1.500.000) o `grande` (más de $1.500.000). Obligatorio solo en cotizaciones.
`subject`, `message`	String	Ver sección 2.
`attachment`	{ `url`, `name`, `size`, `mime` } o null	PDF, JPG o PNG, hasta 2 MB. Tiene que haberse subido con /api/uploads: una URL de otro sitio se rechaza.
`status`	String	`nueva` (por defecto), `en_proceso`, `respondida` o `cerrada`.
`userId`	ObjectId o null	Se completa si quien escribe tiene sesión iniciada. Así se arma «Mis consultas».
`replies`	[{ `text`, `operatorId`, **`operatorName`**, `sentAt` }]	Respuestas anidadas, como en el plan.

visits

Campo	Tipo	Reglas
`path`	String	Página visitada, por ejemplo `/proyectos`.
`userId`	ObjectId o null	Null si es visitante anónimo.
`sessionId`	String	Lo genera el cliente una vez por sesión del navegador.
`createdAt`	Date	Con índice.

La API cuenta una visita por cada par `sessionId` + `path`: recargar la misma página no suma, y dos pedidos iguales a la vez tampoco (lo garantiza un índice único). El contador general es el total de documentos de `visits`.

4. Autenticación y cuenta

Registro, login y Google devuelven el mismo par `token` + `user`; el cliente guarda el token y decide la pantalla de destino según `user.role`: `admin` a `/admin`, `operator` a `/operador`, `user` a la página que pidió o al inicio. Cerrar sesión no tiene endpoint: el cliente borra el token.

Método	Endpoint	Acceso	Qué hace
POST	`/api/auth/register`	Público	Crea una cuenta `user` y abre sesión. 201.
POST	`/api/auth/login`	Público	Abre sesión con email y contraseña.
POST	`/api/auth/google`	Público	Abre sesión con el token de Google; crea la cuenta si no existe.
GET	`/api/auth/me`	Registrado	Devuelve el usuario de la sesión.
PATCH	`/api/auth/me`	Registrado	Cambia `name` y `newsletter`.
POST	`/api/auth/password`	Registrado	Crea la contraseña (cuenta de Google) o la cambia.
DELETE	`/api/auth/me`	Registrado (`user`)	Elimina la cuenta y sus consultas.
POST	`/api/auth/forgot`	Público	Envía el enlace para recuperar la contraseña.
POST	`/api/auth/reset`	Público	Cambia la contraseña con el token del enlace.

Solo registro, login y Google devuelven `{ token, user }`. `GET /api/auth/me` y `PATCH /api/auth/me` devuelven el usuario directamente en `data`.

Registro

```
POST /api/auth/register
{
  "name": "Valentina Ríos",
  "email": "valentina@correo.com",
  "password": "Valentina2026",
  "passwordConfirm": "Valentina2026",
  "acceptTerms": true,
  "newsletter": false
}
```

```
201
{ "ok": true, "data": { "token": "eyJhbGciOi...", "user": { "id":
"6710f0c2a1b2c3d4e5f60718", "name": "Valentina Ríos", "email":
"valentina@correo.com", "role": "user", "active": true, "providers":
["email"], "newsletter": false, "loginCount": 1, "lastLogin": "2026-10-
26T14:02:00.000Z", "createdAt": "2026-10-26T14:02:00.000Z" } } }
```

Si `acceptTerms` no es `true`, la API responde 400 con `fields.acceptTerms`: «Tenés que aceptar para crear la cuenta.». Un email repetido responde 409 `email_en_uso`; el cliente solo lo sabe al enviar, no hay un endpoint para consultar si un email existe. El 409 trae también fields.email con el mismo mensaje, para marcarlo en el campo.

Login y bloqueo

```
POST /api/auth/login
{ "email": "admin@portfolio.test", "password": "Portfolio2026" }
```

La respuesta correcta es igual a la del registro, con 200. Con una contraseña incorrecta:

```
401
{ "ok": false, "error": { "code": "credenciales_invalidas", "message": "Email
o contraseña incorrectos (2 de 5 intentos).", "attemptsLeft": 3 } }
```

Al quinto fallo en 15 minutos para ese email desde la misma IP, la API responde 429 `demasiados_intentos` con `retryAfterSeconds` y el mensaje «Demasiados intentos. Esperá 15 minutos o recuperá tu contraseña.». El mensaje de error es el mismo si el email no existe, para no revelar qué cuentas hay. Un login correcto reinicia el contador de esa IP. El intento se cuenta antes de revisar la clave, así que mandar muchos pedidos en paralelo no da más intentos. Aparte, una IP con 20 logins fallidos queda bloqueada 15 minutos para todas las cuentas, y 50 fallos contra una cuenta desde cualquier IP la bloquean para todos.

Google

```
POST /api/auth/google
{ "credential": "<ID token que entrega Google Identity Services>" }
```

La respuesta suma `isNew: true` si se creó la cuenta, con 201 en ese caso y 200 si ya existía. Si ya había una cuenta con ese email y contraseña, la API le agrega el `googleId` y no crea otra. El nombre de Google se limpia a letras y espacios; si queda de menos de 3 letras se usa la parte local del email. Si el servidor no tiene GOOGLE_CLIENT_ID responde 503 google_no_configurado; si Google no valida el token (o el email no está verificado), 401 no_autenticado; si falta credential, 400 con fields.credential.

Si ya había una cuenta con ese email y contraseña, la API la vincula con Google y **borra esa contraseña** y cierra sus sesiones: el registro no verifica el email, así que quien la creó pudo no ser el dueño. La persona puede crearse una contraseña nueva desde «Mi cuenta». Si la cuenta ya estaba vinculada a OTRA cuenta de Google con el mismo email, responde 403 `sin_permiso`.

Cuenta propia

- `PATCH /api/auth/me` recibe `{ "name": "Valentina Ríos", "newsletter": true }`. El email no se puede cambiar. Sin ninguno de los dos campos responde 400 «No hay nada para guardar.».
- `POST /api/auth/password` recibe `{ "currentPassword", "password", "passwordConfirm" }`. `currentPassword` es obligatorio si la cuenta ya tiene contraseña y no se manda si es una cuenta de Google que la crea por primera vez. Al terminar, la respuesta es `{ "ok": true, "data": { "providers": ["google", "email"] } }`.
- `DELETE /api/auth/me` recibe `{ "confirm": "ELIMINAR" }`, en mayúsculas. Borra la cuenta y todas las consultas con ese `userId`. Un `operator` o un `admin` recibe 403: las cuentas de gestión las borra otro admin desde Usuarios.

Errores de la cuenta propia: si `currentPassword` no coincide, 400 con `fields.currentPassword` «La contraseña actual no es correcta.»; si en `DELETE` no se escribe `ELIMINAR` en mayúsculas, 400 con `fields.confirm` «Escribí ELIMINAR para confirmar.».

Crear o cambiar la contraseña cierra las demás sesiones de la cuenta: la respuesta de `POST /api/auth/password` trae también un `token` nuevo, que el cliente tiene que guardar para no quedar afuera. Lo mismo vale para `/api/auth/reset`: todas las sesiones anteriores dejan de valer y la persona inicia sesión de nuevo.

Recuperar contraseña

1. `POST /api/auth/forgot` con `{ "email" }`. Siempre responde 200 con «Si el email tiene una cuenta, te enviamos un enlace.», exista o no. Máximo 3 pedidos por hora por email.
2. El email lleva el enlace `CLIENT_URL/recuperar?token=<token>`. El token es aleatorio, se guarda con hash y vence en 1 hora. Cuando la página `/recuperar` carga con `?token=`, el cliente salta directo al paso 3 (nueva clave).
3. `POST /api/auth/reset` con `{ "token", "password", "passwordConfirm" }`. Si el token no existe o venció, responde 400 con `code: "token_invalido"` y el mensaje «El enlace venció o ya se usó. Pedí uno nuevo.». Al cambiar la clave, el token se borra y el usuario tiene que iniciar sesión.

`/api/auth/forgot` se limita a 3 pedidos por hora por email: el cuarto responde 429 `demasiados_intentos` con «Pediste demasiados enlaces. Probá de nuevo en una hora.». `/api/auth/reset` valida la contraseña nueva antes de mirar el token: una clave inválida responde 400 `validacion` aunque el enlace sea falso. Un enlace de una cuenta desactivada no se envía. Además hay un límite de 10 pedidos por hora por IP, y el email se manda en segundo plano para que la respuesta tarde igual exista o no la cuenta.

Usuarios desactivados

Un token emitido antes de que un admin desactive la cuenta deja de servir: la API revisa `active` en cada pedido y responde 403 `cuenta_desactivada`.

5. Perfil, proyectos y servicios

Los listados públicos devuelven solo lo visible. Para el admin hay listados aparte (`/api/admin/...`) que incluyen borradores y servicios ocultos, porque la tabla del panel los muestra.

Método	Endpoint	Acceso	Qué hace
GET	`/api/profile`	Público	Perfil del portfolio para la home.
PUT	`/api/profile`	Admin	Guarda el perfil (pantalla Mi perfil). Reemplaza el perfil entero: hay que mandar todos los campos, y los opcionales que falten se vacían.
GET	`/api/projects`	Público	Proyectos visibles, en resumen: destacados primero y después los más nuevos. Filtro `?stack=react`.

Método	Endpoint	Acceso	Qué hace
GET	`/api/projects/:id`	Registrado	Detalle completo. Un borrador solo lo ve el admin; para el resto es 404.
GET	`/api/admin/projects`	Admin	Todos los proyectos con todos los campos, los modificados más recientemente primero. Filtros `?q=` y `?visible=true|false`.
POST	`/api/projects`	Admin	Crea un proyecto. 201.
PUT	`/api/projects/:id`	Admin	Modifica los campos que se envíen; los demás quedan igual.
DELETE	`/api/projects/:id`	Admin	Elimina el proyecto.
GET	`/api/services`	Público	Servicios visibles, ordenados por `order`.
GET	`/api/admin/services`	Admin	Todos los servicios, ordenados.
POST	`/api/services`	Admin	Crea un servicio. Queda al final de la lista. 201.
PUT	`/api/services/:id`	Admin	Modifica los campos que se envíen.
PATCH	`/api/services/order`	Admin	Reordena (flechas arriba y abajo).
DELETE	`/api/services/:id`	Admin	Elimina el servicio.

Los `DELETE` responden `{ "ok": true, "data": { "id": "..." } }`. El cliente pide confirmación antes de llamarlos.

Proyecto: resumen público

Es lo único que ve un visitante. No incluye `demoUrl`, `repoUrl`, `problem`, `solution`, `highlights` ni las imágenes que no son la portada.

```
GET /api/projects
{
  "ok": true,
  "data": [
    { "id": "6710f0c2a1b2c3d4e5f60718", "title": "Gestor de turnos",
"summary": "Reservas online con roles y avisos por email.", "stack":
["React", "Node", "MongoDB"], "cover":
"https://res.cloudinary.com/arce/turnos-1.jpg", "featured": true }
  ],
  "meta": { "page": 1, "limit": 20, "total": 5, "totalPages": 1 }
}
```

Proyecto: detalle (registrado) y alta (admin)

```
GET /api/projects/6710f0c2a1b2c3d4e5f60718
{
  "ok": true,
  "data": {
    "id": "6710f0c2a1b2c3d4e5f60718",
    "title": "Gestor de turnos",
    "summary": "Reservas online con roles y avisos por email.",
    "problem": "Un consultorio de Salta tomaba los turnos por teléfono y en
un cuaderno.",
    "solution": "Una aplicación web donde el paciente elige día y horario y
recibe la confirmación por email.",
    "highlights": ["API REST en Express con validación de cada campo.",
"Autenticación con JWT y roles."],
    "role": "Diseño, backend y frontend",
    "duration": "8 semanas",
    "stack": ["React", "Node", "MongoDB"],
    "images": [ { "url": "https://res.cloudinary.com/arce/turnos-1.jpg",
"publicId": "arce/turnos-1" } ],
    "demoUrl": "https://turnos.arce.dev",
    "repoUrl": "https://github.com/arce/turnos",
    "featured": true,
    "visible": true,
    "updatedAt": "2026-10-20T15:30:00.000Z"
  }
}
```

`POST /api/projects` recibe esos mismos campos sin `id` ni `updatedAt`. Los obligatorios son `title`, `summary` y `stack`; `visible` queda en `false` si no se envía. Para las imágenes, el cliente primero sube cada archivo con `POST /api/uploads` (sección 8) y después manda en `images` los `url` y `publicId` que recibió. La API borra de Cloudinary

las imágenes que dejan de usarse: al sacarlas de un proyecto, al borrar el proyecto y al cambiar la foto del perfil. Solo toca archivos de nuestra cuenta; las URLs de otros sitios (como los placeholders del seed) se ignoran.

Servicio

```
GET /api/services
{
  "ok": true,
  "data": [
    { "id": "6710f1d4a1b2c3d4e5f60801", "name": "Desarrollo web a medida",
"description": "Sitios y aplicaciones completas, del diseño al despliegue.",
"color": "#9B1B22", "priceFrom": 300000, "order": 1 },
    { "id": "6710f1d4a1b2c3d4e5f60804", "name": "Asesoría técnica",
"description": "Revisión de código, arquitectura y buenas prácticas.",
"color": "#4F6B3E", "priceFrom": 50000, "order": 4 }
  ]
}
```

El listado de admin suma `visible` a cada servicio. Para reordenar, el cliente envía la lista completa de ids en el orden nuevo: `PATCH /api/services/order` con `{ "ids": ["...", "..."] }`. La API responde con la lista ya ordenada. Una lista incompleta, con repetidos o con ids que no existen responde 400 con fields.ids. Un nombre repetido (sin distinguir mayúsculas ni tildes) responde 409 nombre_en_uso con fields.name. Al borrar un servicio, los que quedan se renumeran sin huecos.

Perfil

```
PUT /api/profile
{
  "name": "Martina Arce",
  "title": "Desarrolladora Fullstack",
  "phrase": "Productos web de punta a punta.",
  "bio": "Trabajo con pymes, startups y estudios que necesitan un sistema web
completo.",
  "email": "hola@arce.dev",
  "location": "Salta, Argentina · remoto",
  "github": "https://github.com/martinaarce",
  "linkedin": "https://linkedin.com/in/martinaarce",
  "photo": { "url": "https://res.cloudinary.com/arce/martina.jpg",
"publicId": "arce/martina" },
  "stack": ["React", "Node.js", "Express", "MongoDB", "JWT"],
  "experience": [ { "period": "2024 — HOY", "role": "Desarrolladora fullstack
independiente", "detail": "Sistemas a medida para pymes de Salta y Jujuy." }
]
}
```

`GET /api/profile` devuelve los mismos campos en `data`. Si el admin todavía no guardó nada, devuelve el perfil que carga el seed, nunca vacío. PUT reemplaza el perfil entero: GitHub, LinkedIn, foto, stack y experiencia que no se envíen quedan vacíos.

6. Consultas

Una consulta se guarda siempre; el email de confirmación es un efecto aparte. Por eso `POST /api/inquiries` responde 201 apenas queda guardada, y es la señal para mostrar la animación de «enviado» en Contacto.

Método	Endpoint	Acceso	Qué hace
POST	`/api/inquiries`	Público	Guarda la consulta y envía el email de confirmación. 201.
GET	`/api/inquiries`	Operador	Bandeja con filtros y paginación.
GET	`/api/inquiries/mine`	Registrado	Las consultas del usuario con sesión (Mi cuenta).
GET	`/api/inquiries/:id`	Operador	Detalle con el historial de respuestas.
POST	`/api/inquiries/:id/reply`	Operador	Envía la respuesta por email y la guarda.

Método	Endpoint	Acceso	Qué hace
PATCH	`/api/inquiries/:id`	Operador	Cambia el estado.

Enviar una consulta

```
POST /api/inquiries
{
  "name": "Martín Díaz",
  "email": "martin@correo.com",
  "phone": "3874123456",
  "type": "cotizacion",
  "budget": "medio",
  "subject": "Sistema de turnos para mi consultorio",
  "message": "Necesito que mis pacientes saquen turno online y reciban un
recordatorio por email.",
  "attachment": { "url": "https://res.cloudinary.com/arce/brief.pdf", "name":
"brief.pdf", "size": 482113, "mime": "application/pdf" },
  "website": ""
}
```

```
201
{ "ok": true, "data": { "id": "6710f3e8a1b2c3d4e5f60931", "number": 31,
"status": "nueva", "createdAt": "2026-10-15T23:12:00.000Z", "emailSent": true
} }
```

- `phone`, `budget` (salvo en cotizaciones) y `attachment` son opcionales. El adjunto se sube antes con `POST /api/uploads` y aquí solo viaja su descripción. La API solo acepta archivos que subimos nosotros: una URL de otro sitio responde 400 con fields.attachment «El adjunto no es válido. Subilo de nuevo.».
- `website` es el campo oculto anti-spam (honeypot). Si llega con texto, la API responde un 201 falso (con number: 0) y no guarda nada.
- Si el cliente manda el token, la consulta queda asociada a esa cuenta (`userId`). Sin token se guarda igual, como anónima.
- Si el email de confirmación falla, la consulta queda guardada y la respuesta trae `emailSent: false`. El cliente igual muestra el éxito.
- Límite de 5 consultas por hora por IP y de 3 por hora por cada email destinatario (la confirmación se manda a ese email, y así el formulario no sirve para escribirle a un tercero); el sexto responde 429 `demasiados_intentos`.

Bandeja del operador

Parámetros de `GET /api/inquiries`, todos opcionales: `tipo` (`general`, `cotizacion`, `asesoria`), `estado` (`nueva`, `en_proceso`, `respondida`, `cerrada`), `desde` y `hasta` (`YYYY-MM-DD`, ambos días incluidos), `q` (busca en nombre, email y asunto sin distinguir mayúsculas), `page` y `limit`. Orden: más nuevas primero. Un valor inválido en un filtro responde 400 con un mensaje por campo. Las fechas son de Salta.

```
GET /api/inquiries?tipo=cotizacion&estado=nueva&page=1&limit=20
{
  "ok": true,
  "data": [
    { "id": "6710f3e8a1b2c3d4e5f60931", "number": 31, "name": "Martín Díaz",
"email": "martin@correo.com", "type": "cotizacion", "subject": "Sistema de
turnos para mi consultorio", "status": "nueva", "repliesCount": 0,
"createdAt": "2026-10-15T23:12:00.000Z" }
  ],
  "meta": { "page": 1, "limit": 20, "total": 14, "totalPages": 1 }
}
```

Detalle y respuesta

`GET /api/inquiries/:id` devuelve todos los campos del modelo: `phone`, `budget`, `message`, `attachment` y `replies`. Abrir el detalle no cambia el estado.

```
POST /api/inquiries/6710f3e8a1b2c3d4e5f60931/reply
{ "text": "Hola Martín, gracias por escribir. Te propongo una reserva online
por día y profesional, con recordatorio por email." }
```

```
200
{ "ok": true, "data": { "id": "6710f3e8a1b2c3d4e5f60931", "number": 31,
"status": "respondida", "replies": [ { "text": "Hola Martín, gracias por
escribir...", "operatorId": "6710f0c2a1b2c3d4e5f60700", "operatorName":
"Operador Portfolio", "sentAt": "2026-10-16T10:05:00.000Z" } ] } }
```

La API manda el email primero. Si sale bien, guarda la respuesta y pasa el estado a `respondida`. Si el email falla, no guarda nada, no cambia el estado y responde 502 `email_no_enviado`, así el operador puede reintentar sin duplicar la respuesta. Después de responder, el operador puede volver a cambiar el estado, por ejemplo a `cerrada`.

`PATCH /api/inquiries/:id` recibe `{ "status": "en_proceso" }` y devuelve la consulta actualizada.

Mis consultas

```
GET /api/inquiries/mine
{
  "ok": true,
  "data": [
    { "id": "6710f3e8a1b2c3d4e5f60931", "number": 31, "type": "cotizacion",
"subject": "Tienda online para mi emprendimiento", "status": "respondida",
"createdAt": "2026-10-12T14:00:00.000Z", "lastReply": { "text": "Hola
Valentina, gracias por escribir...", "sentAt": "2026-10-13T09:30:00.000Z" } }
  ]
}
```

`lastReply` es `null` si todavía no hay respuestas. Solo aparecen las consultas con el `userId` de la sesión; una consulta enviada sin iniciar sesión no se vincula a ninguna cuenta aunque el email coincida. Devuelve como máximo las 100 más nuevas.

7. Usuarios, visitas y estadísticas

Gestión de usuarios (admin)

Método	Endpoint	Acceso	Qué hace
GET	`/api/users`	Admin	Lista con `?rol=user|operator|admin`, `?q=` (nombre o email), `page` y `limit`.
GET	`/api/users/:id`	Admin	Un usuario.
PATCH	`/api/users/:id`	Admin	Cambia `role` y/o `active`.
DELETE	`/api/users/:id`	Admin	Elimina la cuenta y sus consultas.

Cada usuario del listado tiene los campos de la sección 3: `name`, `email`, `role`, `active`, `providers`, `loginCount`, `lastLogin` y `createdAt`, que son las columnas de la tabla del panel.

`PATCH` acepta solo `role` y `active`: el nombre y el email no se cambian desde acá. Sin ninguno de los dos responde 400 «No hay nada para guardar.»; un rol o un valor inválido, 400 con `fields.role` («Rol no válido.») o `fields.active` («Valor no válido.»). El listado va de los más nuevos a los más viejos. Un admin sí puede degradar, desactivar o eliminar a otro admin. `DELETE` borra la cuenta y también sus consultas y sus accesos; las visitas se conservan, sin dueño.

```
PATCH /api/users/6710f0c2a1b2c3d4e5f60718
{ "role": "operator" }
```

La API rechaza con 403 `sin_permiso` estos casos, con el mensaje listo para mostrar:

- El admin se desactiva a sí mismo: «No podés desactivar tu propia cuenta de administrador.»
- El admin se quita el rol a sí mismo: «No podés quitarte el rol de administrador.»
- El admin se elimina a sí mismo: «No podés eliminar tu propia cuenta.»

La primera versión listaba un cuarto caso, «Tiene que quedar al menos un administrador activo.». No existe como regla aparte: con estas tres siempre queda un admin activo, porque quien hace el pedido es un admin activo y, si el cambio es sobre otro, él sigue estando.

Con la decisión 2, el mensaje de «cuenta desactivada» del prototipo («Ya no puede ver las demos») tiene que cambiar a «Ya no puede iniciar sesión».

Registrar una visita

```
POST /api/visits
{ "path": "/proyectos", "sessionId": "b3f6c1de-8a4e-4c8e-9d27-5a0f1e2b7c90" }
```

Responde `{ "ok": true, "data": { "counted": true } }`. Si ese `sessionId` ya registró ese `path`, responde `counted: false` y no suma. Si el cliente manda el token, la visita guarda el `userId`. Las rutas que empiezan con `/admin` o `/operador` no se cuentan. B genera el `sessionId` una vez por sesión del navegador y lo guarda en `sessionStorage`.

Responde 201 la primera vez que cuenta y 200 con `counted: false` las demás. `path` tiene que empezar con `/` (se guarda sin `?consulta` ni `#ancla`, hasta 200 caracteres) y `sessionId` ser de 8 a 100 letras, números, guiones o guiones bajos; si no, 400 con `fields`. El límite es de 60 pedidos por minuto por IP.

Estadísticas del dashboard (admin)

Los cuatro endpoints aceptan `desde` y `hasta` (`YYYY-MM-DD`, ambos incluidos). Si no se mandan, se usan los últimos 14 días; el máximo es un rango de 366 días. Los días se cuentan en hora de Salta (UTC−3), no en UTC, para que el gráfico coincida con lo que ve el admin.

Método	Endpoint	Parámetros extra	Alimenta
GET	`/api/stats/summary`	`tipo` (opcional)	Las cuatro tarjetas.

Método	Endpoint	Parámetros extra	Alimenta
GET	`/api/stats/visits`	`agrupar=dia|semana`	Gráfico de visitas por día o por semana.
GET	`/api/stats/inquiries-by-type`	ninguno	Barras de consultas por tipo.
GET	`/api/stats/registrations`	`agrupar=semana`	Registros por semana (si no hay rango: últimas 6 semanas).

```
GET /api/stats/summary?desde=2026-10-02&hasta=2026-10-15&tipo=cotizacion
{
  "ok": true,
  "data": {
    "visits": 779,
    "users": { "total": 40, "newThisMonth": 6 },
    "logins": 312,
    "inquiries": 14
  }
}
```

Todo son números simples, listos para `<NumeroAnimado valor={n} />`. `visits` y `logins` son del período; `users.total` es el total histórico; `users.newThisMonth` cuenta el mes calendario en curso; `inquiries` son las del período y, si se manda `tipo`, solo de ese tipo.

```
GET /api/stats/visits?desde=2026-10-02&hasta=2026-10-04&agrupar=dia
{ "ok": true, "data": [ { "date": "2026-10-02", "count": 38 }, { "date":
"2026-10-03", "count": 0 }, { "date": "2026-10-04", "count": 51 } ] }
```

```
GET /api/stats/inquiries-by-type?desde=2026-10-02&hasta=2026-10-15
{ "ok": true, "data": [ { "type": "cotizacion", "count": 14 }, { "type":
"general", "count": 9 }, { "type": "asesoria", "count": 7 } ] }
```

- Las series vienen siempre completas: los días o semanas sin datos aparecen con `count: 0`. Por eso el gráfico no tiene huecos.
- Con `agrupar=semana`, `date` es el lunes de esa semana. Un rango que corta una semana a la mitad suma solo los días incluidos.
- `inquiries-by-type` devuelve siempre los tres tipos, aunque alguno tenga 0.
- `GET /api/stats/registrations` devuelve el mismo formato que `visits`, con la cantidad de cuentas creadas por semana.

Una fecha inválida, un rango invertido, uno de más de 366 días, un `agrupar` o un `tipo` desconocidos responden 400 con un mensaje por campo, todos juntos. Los últimos 14 días incluyen hoy (hoy y los 13 anteriores), y la última semana de una serie semanal es parcial hasta que termina. `registrations` sin rango trae 6 semanas completas, desde el lunes de hace 5 semanas, y también acepta `agrupar=dia`.

8. Archivos, emails y datos de prueba

Subida de archivos

Un solo endpoint sirve para las imágenes de proyectos, la foto del perfil y el adjunto del contacto. Los archivos se guardan en Cloudinary, porque el disco de Render gratis se borra en cada reinicio.

```
POST /api/uploads        (multipart/form-data)
campos: file = el archivo, kind = project | profile | attachment
```

`kind`	Quién sube	Formatos	Peso máximo
`project`	Admin	JPG, PNG, WebP	2 MB
`profile`	Admin	JPG, PNG, WebP	2 MB
`attachment`	Público	PDF, JPG, PNG	2 MB, y hasta 3 subidas por hora por IP

```
201
{ "ok": true, "data": { "url":
"https://res.cloudinary.com/arce/image/upload/v1/turnos-1.jpg", "publicId":
"arce/turnos-1", "name": "turnos-1.jpg", "size": 318442, "mime": "image/jpeg"
} }
```

La API revisa el contenido real del archivo, no solo la extensión. Un formato fuera de la lista responde 415 y un archivo de más de 2 MB responde 413; ambos traen `fields.file` con el mensaje de la sección 2. Recomiendo que el cliente suba el archivo apenas el usuario lo elige: así el error aparece junto al selector y la vista previa usa la URL definitiva. Un archivo subido que nunca se asocia a un proyecto queda huérfano en Cloudinary; es aceptable para este trabajo.

Otros errores de `POST /api/uploads`: sin sesión en `project` o `profile`, 401 `no_autenticado`; con sesión pero sin ser admin, 403 `sin_permiso`; sin archivo, 400 «Elegí un archivo.»; un `kind` desconocido, 400 con `fields.kind` «Tipo de subida no válido.»; un archivo de más de 5 MB se corta y responde 413 «El archivo pesa más de 5 MB. El máximo es 2 MB.». El límite de 3 subidas por hora por IP vale para todos menos el admin, incluido el

operador. `name` no lleva carpetas ni HTML, y los PDF se guardan en Cloudinary como `raw`. Si Cloudinary falla, 502 `almacenamiento_error`; si el servidor no tiene credenciales, 503 `almacenamiento_no_configurado`.

Emails

Salen por Nodemailer. En tu PC, desde una cuenta de Gmail con contraseña de aplicación; en Render gratis, desde un servicio SMTP con el puerto 2525, porque Render bloquea los puertos 25, 465 y 587 y Gmail no funciona ahí (ver DEPLOY.md). Gmail no deja enviar como `hola@arce.dev`, un dominio ficticio. El remitente real es la cuenta del proyecto, con el nombre «Martina Arce · arce.dev». El `hola@arce.dev` de las maquetas es solo ilustrativo.

Email	Cuándo sale	Asunto	Contenido
Confirmación	Al guardar una consulta	«Recibimos tu consulta · Cotización»	N.º de consulta, saludo con el nombre, resumen (tipo, asunto, mensaje, fecha) y botón a Proyectos.
Respuesta	Cuando el operador responde	«Re: <asunto>»	Texto del operador, mensaje original y botón «Ver en mi cuenta» si la consulta tiene `userId`.
Recuperar contraseña	Al pedir el enlace	«Recuperá tu contraseña»	Botón con el enlace, que vence en 1 hora. Existe con un diseño provisional, de texto simple; A debe diseñarlo.

Dos textos de las maquetas se cambiaron en las plantillas, porque no se podían cumplir. A tiene que confirmarlos:

- El email de respuesta dice «Respondé este email para seguir la conversación. Queda guardado en el historial de tu consulta.» La API no lee emails entrantes. Quedó así: «Para seguir la conversación, escribinos a la dirección de este mensaje. Tu respuesta no queda en el historial.»
- El email de confirmación dice «No respondas a este mensaje» y el de respuesta invita a responder. Ahora ambos usan el mismo criterio: «escribinos a la dirección de este mensaje». El `Reply-To` de los dos es la cuenta del proyecto.

Las plantillas viven en `server/utils/emailTemplates.js`, con HTML de 600 px de ancho y estilos en línea, como el prototipo. Los botones: «Responder» abre un email a la cuenta del proyecto (solo si hay MAIL_USER configurado) y «Ver en mi cuenta» aparece solo si la

consulta tiene userId. Todo texto que escribe una persona se escapa en el HTML.

Seed

`npm run seed` en `server/` borra y recarga la base con datos que reproducen el prototipo. Con eso el dashboard se ve completo en la exposición. Las fechas son relativas al día en que se corre.

Colección	Qué carga
`users`	40 cuentas: 1 admin, 1 operador y 38 usuarios. Entre los usuarios están Valentina Ríos (Google), Lucía Gómez, Mateo Fernández, Julieta Sosa (desactivada) y Nicolás Pérez, como en la pantalla Usuarios. 6 se crean en el mes en curso.
`projects`	6: Gestor de turnos, Dashboard de ventas, Tienda online, Clima en vivo, Landing animada y Blog con CMS (este queda como borrador, sin demo). Las imágenes son placeholders de placehold.co hasta que haya fotos reales en Cloudinary.
`services`	4: Desarrollo web a medida ($300.000), APIs REST ($200.000), Bases de datos ($150.000) y Asesoría técnica ($50.000), con los colores del prototipo.
`inquiries`	30 en los últimos 30 días: 14 de cotización, 9 generales y 7 de asesoría, con los cuatro estados y respuestas en las respondidas. Numeradas del 1 al 30. Dos son de Valentina Ríos (una respondida y otra en proceso): las que muestra «Mi cuenta».
`visits`	Entre 24 y 72 por día durante 30 días.
`logins`	Unos 22 por día, para que «accesos autenticados» tenga datos.
`profile`	El perfil de Martina Arce del prototipo. Los años y el puesto de la experiencia, que el prototipo dejaba como [AÑO] y [EMPRESA], están inventados: se editan desde Mi perfil.

Usuarios de prueba para la profesora

Rol	Email	Contraseña
Administrador	`admin@portfolio.test`	La define `SEED_PASSWORD`
Operador	`operador@portfolio.test`	La define `SEED_PASSWORD`
Usuario registrado	`usuario@portfolio.test`	La define `SEED_PASSWORD`

En desarrollo, si `SEED_PASSWORD` no está definida, el seed usa `Portfolio2026`, la misma que usa el login simulado del prototipo. En producción el seed se niega a correr sin esa variable, porque el backend desplegado es público y esas cuentas dan acceso de administrador. La contraseña real se le pasa a la profesora por fuera del repositorio.

Valentina Ríos y Mateo Fernández son cuentas de Google simuladas: no se puede iniciar sesión con ellas. Por eso «Mi cuenta» de `usuario@portfolio.test` sale sin consultas. Para mostrar «Mis consultas» en la demo, hay que enviar una consulta con esa sesión iniciada.

9. Qué tiene que hacer cada uno

Este contrato pide cambios chicos en el prototipo y en el código de B y D. Los listo por persona para que nadie lo descubra al integrar.

A (diseño)

- Corregir en el manual que el admin también entra a `/operador` (decisión 3).
- Cambiar en la pantalla Usuarios el mensaje de cuenta desactivada: «Ya no puede iniciar sesión».
- Confirmar los dos textos de los emails que ya se cambiaron (sección 8) y diseñar el email de recuperación de contraseña, que hoy es de texto simple.
- Agregar el máximo de 120 caracteres al asunto del formulario de contacto.
- Fuera de la API: el plan dice Tailwind y el manual dice Bootstrap 5; hay que cerrar cuál es.

B (frontend público)

- Mandar el token en todos los pedidos como `Authorization: Bearer ...`, también en `POST /api/inquiries` y `POST /api/visits` cuando haya sesión.
- Registro: enviar `passwordConfirm` y `acceptTerms`. Mostrar los errores de `error.fields` junto a cada campo.

- Contacto: enviar `website` vacío (campo oculto), subir el adjunto antes con `POST /api/uploads` y mostrar `<EnviadoLottie />` al recibir el 201. Aunque `emailSent` sea `false`, se muestra el éxito.
- Proyectos: mostrar `<GuardaLoader />` mientras responde `GET /api/projects`. El listado trae `cover`, no un arreglo de imágenes.
- Login: manejar `attemptsLeft` (401) y `retryAfterSeconds` (429). Google: enviar `credential`.
- `/recuperar`: si la URL trae `?token=`, saltar al paso 3 y llamar a `POST /api/auth/reset`.
- Visitas: generar un `sessionId` por sesión del navegador y llamar a `POST /api/visits` en cada cambio de ruta pública.
- La home lee el perfil de `GET /api/profile` en vez de tener los textos fijos.
- Mi cuenta: `GET` y `PATCH /api/auth/me` devuelven el usuario directo en `data` y `DELETE` devuelve `{ id }`. La contraseña se crea o cambia con `POST /api/auth/password` (y guardá el token nuevo que devuelve, porque las demás sesiones se cierran) y las consultas salen de `GET /api/inquiries/mine`.
- Todos los 429 traen `retryAfterSeconds`: mostrar cuánto falta en contacto, subida de adjuntos y recuperación de contraseña.

D (paneles)

- Bandeja: los filtros se llaman `tipo`, `estado`, `desde`, `hasta` y `q`, más `page` y `limit`. Los listados traen `meta.total` y `meta.totalPages`. El adjunto de una consulta es un enlace de Cloudinary.
- Proyectos: el formulario manda `stack` como lista (hay que separar por comas y recortar), y suma los campos `problem`, `solution`, `highlights`, `role` y `duration` que el prototipo todavía no tiene. El listado del panel es `GET /api/admin/projects`. Las imágenes se suben antes con `kind=project`; al sacar una, la API la borra de Cloudinary.
- Servicios: el color reemplaza al ícono; el orden se guarda con `PATCH /api/services/order`; el listado del panel es `GET /api/admin/services`.
- Usuarios: `GET /api/users?rol=&q=&page=&limit=`, `PATCH` con `role` y/o `active`, y `DELETE`, que borra también las consultas de esa cuenta. Usar los mensajes de la API tal cual cuando responda 403.
- Mi perfil: `GET` y `PUT /api/profile`; la foto se sube antes con `kind=profile`. `PUT` reemplaza todo, así que hay que mandar todos los campos; GitHub y LinkedIn son opcionales.

- Dashboard: los `KPI` son números simples para `<NumeroAnimado valor={n} />`. Las series ya vienen completas, con ceros. `summary` acepta `tipo`, las fechas del filtro se mandan como `AAAA-MM-DD` (hora de Salta) y la última semana de una serie semanal es parcial.

C (backend)

Orden de entrega propuesto, para que B y D puedan integrar sin esperar al final:

Fecha objetivo	Qué queda funcionando	Desbloquea a
28/10	Servidor, modelos, formato de errores, seed, auth (registro, login, Google, `me`) y lecturas públicas de perfil, proyectos y servicios	B
30/10	Consultas con emails, bandeja del operador y subida de archivos	B (Contacto) y D (Bandeja)
1/11	CRUD de admin (proyectos, servicios, perfil) y backend desplegado en Render	D
4/11	Cuenta propia, recuperar contraseña, usuarios de admin, visitas y estadísticas	B (Mi cuenta) y D (Usuarios, Dashboard)
6/11	Límites de intentos, revisión de seguridad y documentación de la API	Todos

Estado al 26/10: está implementado y probado en local todo lo de las filas del 28/10, 30/10, 1/11 (salvo el deploy en Render) y 4/11, y también los límites de intentos de la fila del 6/11. Faltan el deploy en Render, la revisión de seguridad final y el documento de arquitectura. Las fechas siguen siendo una propuesta; la exposición cierra el 13/11.

Preguntas: qué se resolvió y qué sigue abierto

- Resuelto: la casilla de newsletter se guarda en `newsletter`, pero la API no envía ningún email.
- Resuelto: la sesión dura 7 días; se cambia con la variable `JWT_EXPIRES_IN` del servidor.
- Resuelto: al borrar un usuario se borran también sus consultas, tanto desde «Eliminar mi cuenta» como desde Usuarios.
- Abierto: una cuenta desactivada no puede iniciar sesión (decisión 2). Falta que lo confirmen A y D; el comentario de la sección 1 sigue esperando respuesta.

10. Qué cambió desde la primera versión

Esta tabla junta lo que difiere de la versión del 26/10 al implementarla, para que B y D vean de un vistazo qué revisar. Cada cambio está explicado en su sección.

Cambio	Detalle	Afecta a
`me` devuelve el usuario directo	`GET` y `PATCH /api/auth/me` devuelven el usuario en `data`, no `{ user }`. `DELETE` devuelve `{ id }`.	B
Orden de los proyectos públicos	Primero los destacados y después los más nuevos.	B
GitHub y LinkedIn opcionales	El prototipo los exige; la API acepta vacío y los guarda en `null`.	D
`PUT /api/profile` reemplaza todo	Los campos opcionales que no se mandan se vacían.	D
Códigos de error nuevos	`almacenamiento_no_configurado` (503), `almacenamiento_error` (502), `google_no_configurado` (503) y `pedido_muy_grande` (413). `token_invalido` y `email_no_enviado` pasaron a la tabla de la sección 2.	B, D
`fields` en los 409	`email_en_uso` y `nombre_en_uso` traen `fields` con el mismo mensaje.	B, D
Regla del último admin	Se eliminó el cuarto caso de la sección 7: con las tres reglas siempre queda un admin activo.	D
Adjuntos de consulta	Solo se aceptan archivos subidos con `POST /api/uploads`; una URL ajena se rechaza.	B
Nombre y asunto de la consulta	Los saltos de línea y los espacios repetidos se juntan en uno.	B
Imágenes en Cloudinary	La API las borra al sacarlas de un proyecto, al borrar el proyecto y al cambiar la foto del perfil.	D
Visitas	201 si cuenta y 200 si no; los paneles no cuentan; un índice único evita duplicados.	B
Estadísticas	Los días son de Salta, los últimos 14 días incluyen hoy y la última semana es parcial. `registrations` sin rango trae 6 semanas.	D

Cambio	Detalle	Afecta a
Límites de pedidos	5 consultas por hora, 3 subidas por hora, 60 visitas por minuto, 3 recuperaciones por hora y 5 logins fallidos en 15 minutos. Ver la decisión 14.	B
Textos de los emails	Los textos corregidos de la sección 8 ya están aplicados; el email de recuperación es provisional.	A
Colecciones internas	`counters` y `loginattempts`, además de las siete del contrato.	C
Datos de prueba	Imágenes de placehold.co, experiencia inventada, y Valentina y Mateo sin login real.	C, D
Contraseña y sesiones	POST /api/auth/password devuelve también un token nuevo, que el cliente tiene que guardar. Cambiar o recuperar la contraseña cierra todas las demás sesiones.	B
Login con Google	Si la cuenta ya existía con contraseña, se vincula y se borra esa contraseña (el registro no verifica el email). Si ya estaba vinculada a otra cuenta de Google, responde 403.	B
Bloqueo de login	Es por email y por IP, con un tope de 50 por cuenta y de 20 fallos por IP. El intento se cuenta antes de revisar la clave.	B
Revisión de seguridad	16 correcciones. El detalle y los riesgos que quedan están en SECURITY.md del repositorio.	Todos

Variables de entorno del servidor

Variable	Para qué sirve	Obligatoria
`MONGODB_URI`	Conexión a MongoDB Atlas.	Sí
`JWT_SECRET`	Clave con la que se firman los tokens.	Sí

Variable	Para qué sirve	Obligatoria
`CLIENT_URL`	URL del cliente, para CORS y para los enlaces de los emails. Admite varias separadas por coma.	En producción (por defecto `http://localhost:5173`)
`PORT`	Puerto de la API.	No (por defecto 4000)
`JWT_EXPIRES_IN`	Duración de la sesión.	No (por defecto `7d`)
`GOOGLE_CLIENT_ID`	Login con Google. Sin esto, `/api/auth/google` responde 503.	Para usar Google
`MAIL_USER` y `MAIL_PASS`	Usuario y clave del servicio de emails (Gmail con contraseña de aplicación solo sirve en tu PC). Sin esto, los emails salen por la consola del servidor.	Para enviar emails
`CLOUDINARY_URL` (o `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY` y `CLOUDINARY_API_SECRET`)	Subida de archivos. Sin esto, `/api/uploads` responde 503.	Para subir archivos
`SEED_PASSWORD`	Contraseña de las cuentas de prueba. En producción el seed no corre sin ella.	Para el seed
`NODE_ENV`	`production` en Render.	Recomendada
MAIL_HOST y MAIL_PORT	Servidor y puerto SMTP. Sin MAIL_HOST se usa Gmail. En Render gratis: el servidor de tu servicio SMTP y el puerto 2525.	En Render

Variable	Para qué sirve	Obligatoria
MAIL_FROM	Email remitente y al que responden. Con Gmail alcanza con MAIL_USER; con otros servicios el usuario SMTP no es un email, así que va aparte (y tiene que estar verificado en el servicio).	Con un SMTP que no es Gmail
TRUST_PROXY	Cuántos proxies hay delante del servidor, para que los límites de intentos cuenten por visitante. Si todos comparten el límite, probar con 2.	No (por defecto 1)